

制御文

制御文とは

制御文とは、プログラムの実行順番を制御するものです。

いままでのプログラムは必ず上から順番に1行ずつ実行されていました。制御文を使うことで、実行するしない、どれを実行する、何回実行する、などの要素を調整することができます。

代表的なものに下記があります。

- if switch
- for foreach while

if文

別名を条件分岐といいます。
条件を満たした場合のみ実行するプログラムを作成することで、
ユーザーの行動によって処理を変えたり、
設定によって処理を変えたりできます。

if文

```
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20
```

0 個の参照

```
static void Main(string[] args)  
{  
    int number = 1;  
  
    if (number == 1)  
    {  
        Console.WriteLine("Numberは1です。");  
    }  
    else  
    {  
        Console.WriteLine("Numberは1以外です。");  
    }  
}
```

if文

解説

if(条件)の比較の仕方はいくつかあります。

これ以外にもありますが、下記が代表的なものです。

== 中身が同じ

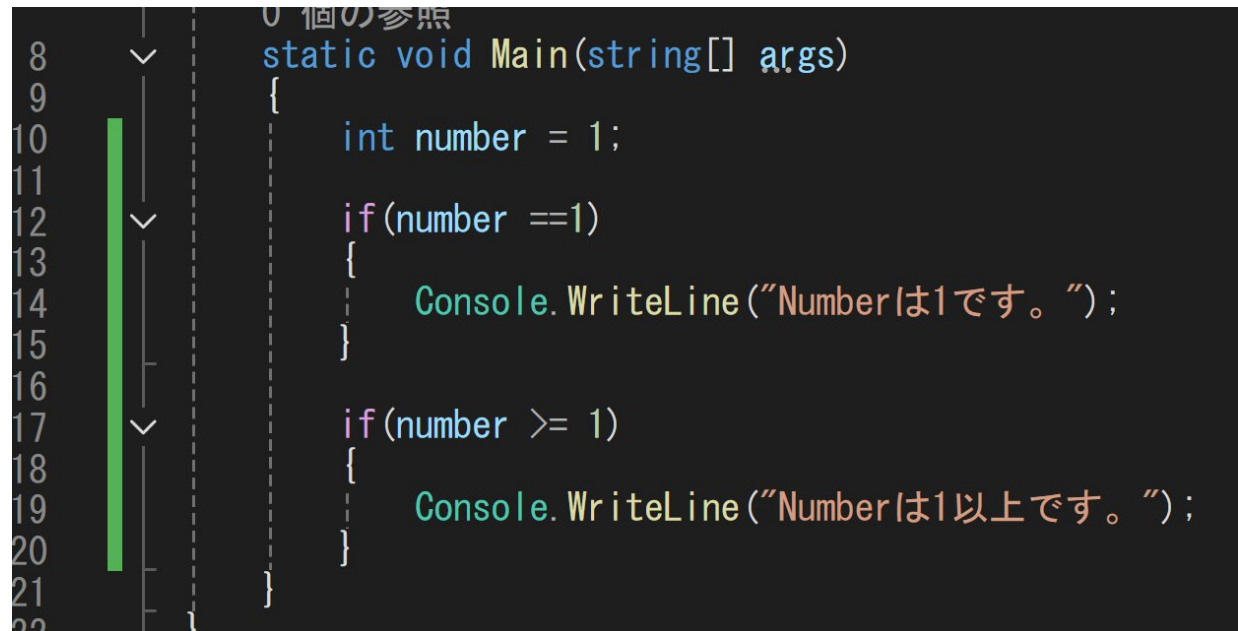
!= 中身が違う

>= 左側が右側以上

<= 左側が右側以下

> 左側が右側より大きい

< 左側が右側より小さい

A screenshot of a code editor with a dark background. On the left, there is a vertical scrollbar and line numbers from 8 to 22. The code is in C# and shows a static void Main method. Inside the method, an integer variable 'number' is initialized to 1. There are two if statements. The first if statement checks 'if(number == 1)' and calls 'Console.WriteLine("Numberは1です。");'. The second if statement checks 'if(number >= 1)' and calls 'Console.WriteLine("Numberは1以上です。");'.

```
8  static void Main(string[] args)
9  {
10     int number = 1;
11
12     if(number == 1)
13     {
14         Console.WriteLine("Numberは1です。");
15     }
16
17     if(number >= 1)
18     {
19         Console.WriteLine("Numberは1以上です。");
20     }
21 }
22
```

Switch文

解説

if文の派生にswitch文（多岐分岐、選択分岐）という機能があります。
こちらは複数の条件で簡単に分岐させる事ができます。

実行例

```
6 void main()
7 {
8     int number = 1;
9     string oneText = "numberの設定は1です。";
10    string twoOrThreeText = "numberの設定は2か3です";
11    string otherText = "numberの設定はそれ以外です。";
12
13    switch (number)
14    {
15        case 1:
16            printf(oneText.c_str());
17            break;
18        case 2:
19        case 3:
20            printf(twoOrThreeText.c_str());
21            break;
22        default:
23            printf(otherText.c_str());
24            break;
25    }
26
27 }
```

複雑な条件＜連続したif文＞

解説

if文は連続して書くことができます。

書くことはできますが、長くなれば長くなるほど分かりにくくなるので、出来る限り避けるべきではあります。

実行例

```
6 void main()
7 {
8     int number = 1;
9     string pulsText = "正の数です。";
10    string minusText = "負の数です。";
11    string oneText = "1です";
12    string otherText = "それ以外です。";
13
14    if (number > 0)
15    {
16        printf(pulsText.c_str());
17    }
18    else if (number < 0)
19    {
20        printf(minusText.c_str());
21    }
22    else if (number == 1)
23    {
24        printf(oneText.c_str());
25    }
26    else
27    {
28        printf(otherText.c_str());
29    }
30 }
31
```

複雑な条件＜かつ、または（論理積、論理和）＞

解説

条件は複雑にする事ができます。

これも、必要以上に複雑になる可能性があるので、
極力使わないようにする事が多いです。

実行例

```
6 void main()
7 {
8     int number = 1;
9     string oneUpThreeDownText = "1以上 3 以下です。";
10    string fourOrFive = "4か5です。";
11    string otherText = "それ以外です。";
12
13    if (number >= 1 && number <= 3)
14    {
15        printf(oneUpThreeDownText.c_str());
16    }
17    else if( number == 4 || number == 5)
18    {
19        printf(fourOrFive.c_str());
20    }
21    else
22    {
23        printf(otherText.c_str());
24    }
25 }
```


複雑な条件<if文のネスト（2重構造）>

解説

if文の中にもif文を書く事ができます。

これも複雑になるので余り推奨はされません。

実行例

```
6 void main()
7 {
8     int number = 1;
9     string zeroUpText = "正の数です";
10    string oneText = "1です。";
11
12    if (number > 0)
13    {
14        printf(zeroUpText.c_str());
15
16        if (number == 1)
17        {
18            printf(oneText.c_str());
19        }
20    }
21 }
```

while・for文

別名を繰り返し処理といいます。
同じ事を何度も行いう事ができます。
この際に、繰り返す回数を指定する事や、
終わる条件を決める事ができます。

while文

```
static void Main(string[] args)
{
    while (true)
    {
        string input = Console.ReadLine();
        if(input == "1")
        {
            Console.WriteLine("終了する");
            break;
        }
        else
        {
            Console.WriteLine("ループする");
        }
    }
}
```

for文

0 個の参照

```
class Program
```

```
{
```

0 個の参照

```
static void Main(string[] args)
```

```
{
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        Console.WriteLine(i + "回目");
```

```
    }
```

```
}
```

```
}
```

for文とwhile文の使い分け

for文は指定の回数同じ処理をする場合に使います。
while文は指定の条件が達成される場合に使います。

用途によって使い分けられるようになるとよいですが、
基本はfor文をよく使います。

稀に使うやつ <do while文>

do while文は必ず一回は動作するwhile文です。

通常のwhile文は条件が成立している場合に実行されますが、do while文は条件が成立していなくても1回は実行されます。

```
6 void main()
7 {
8     string startText = "10個の配列の中身を表示します。¥n";
9     string nowText = "現在は%d回目です¥n";
10    string endText = "終了します。¥n";
11
12    int i = 0;
13
14    // 開始の合図を出す
15    printf(startText.c_str());
16
17    // 繰り返し処理開始
18    do
19    {
20        // 回数追加
21        i++;
22
23
24        // 繰り返し回数の表示
25        printf(nowText.c_str(), i);
26
27    } while (false); // 必ずループしない
28
29
30    // 終了の合図を出す
31    printf(endText.c_str());
32 }
```

10個の配列の中身を表示します。
現在は1回目です
終了します。

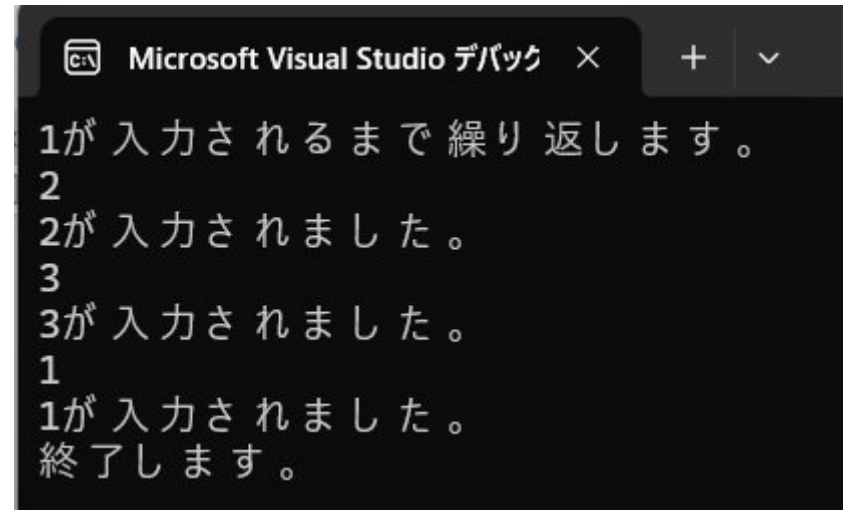
稀に使うやつ <無限ループ>

while(true)を使う事で無限にループできます。

break;を呼び出すとループが終了します。

処理の最初で条件が決まらない場合に使います。

```
6 void main()
7 {
8     string startText = "1が入力されるまで繰り返します。¥n";
9     string continueText = "%sが入力されました。¥n";
10    string endText = "終了します。¥n";
11    string input;
12
13    // 開始の合図を出す
14    printf(startText.c_str());
15
16    // 無限ループ開始
17    while (true)
18    {
19        // 入力受付
20        getline(cin, input);
21
22        // 入力された文字を表示
23        printf(continueText.c_str(), input.c_str());
24
25        // 入力された文字が1か判定
26        if (input == "1")
27        {
28            // 無限ループ終了
29            break;
30        }
31    }
32
33    printf(endText.c_str());
34 }
35
```



```
Microsoft Visual Studio デバック × + v
1が入力されるまで繰り返します。
2
2が入力されました。
3
3が入力されました。
1
1が入力されました。
終了します。
```

繰り返し処理 <配列>

解説

配列は繰り返し処理とはちょっと違いますが、一緒に使われる事が多いので一緒に紹介します。変数名の後ろに[]をつけるとた変数をたくさん保存する事ができます。

イメージ

```
int number;
```



```
int number[10];
```



繰り返し処理 <配列>

実行例

```
7 static void Main(string[] args)
8 {
9     // 配列の作成
10    int[] numbers = new int[10];
11
12    //配列の中にデータを入れる
13    for(int i = 0; i < 10; i++)
14    {
15        numbers[i] = i;
16    }
17
18    //配列の中身を表示する
19    for(int i = 0; i < 10; i++)
20    {
21        Console.WriteLine(numbers[i]);
22    }
23 }
```

Microsoft Visual Studio デバッグ × + ▾

```
0
1
2
3
4
5
6
7
8
9
```

繰り返し処理 <配列>

```
// 配列の宣言のしかた
```

```
//要素数が分かるけど、中身が未確定の場合
```

```
int count = 10;
```

```
int[] numbers = new int[count];
```

```
// 中身が何か確定している場合
```

```
int[] numbers2 = {0, 1, 2, 3, 4, 5, 6 };
```

繰り返し処理 <配列>

```
// パイザでよくある配列の作り方
int length = 5; // 入力で何個のデータかもらえる
int[] data = new int[length];
for (int i = 0; i < length; i++)
{
    string text = Console.ReadLine();
    data[i] = int.Parse(text);
}
```

入力例1

5 50
23
5
14
6
9

繰り返し処理 <配列>

よく見るこいつも配列を作っています。
入力から受け取った文字をスペース区切りで配列の中に保存しています。

```
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        string text = Console.ReadLine();
        int[] numbers = text.Split(" ").Select(int.Parse).ToArray();
    }
}
```